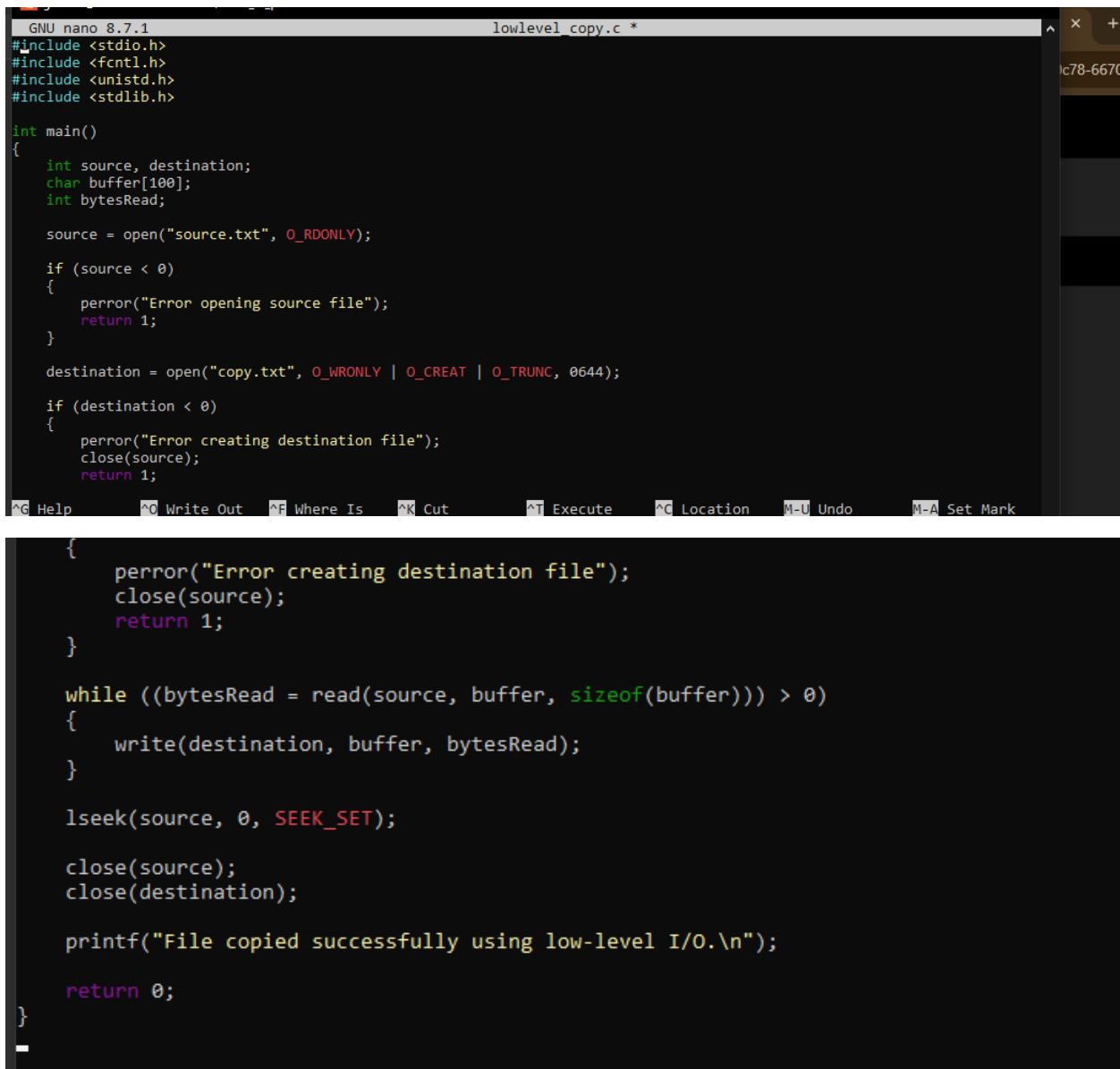


Practical week – 9

```
ganesh@DESKTOP-79N3772:~$ pwd
/home/ganesh
ganesh@DESKTOP-79N3772:~$ mkdir -p ~/linux_io_practical
ganesh@DESKTOP-79N3772:~$ cd ~/linux_io_practical
ganesh@DESKTOP-79N3772:~/linux_io_practical$ pwd
/home/ganesh/linux_io_practical
ganesh@DESKTOP-79N3772:~/linux_io_practical$
```

Creating a practical folder



The image shows a terminal window with a dark background. The top part shows the user creating a directory and changing to it. The bottom part shows a nano editor editing a C file named 'lowlevel_copy.c'. The code in the editor is for a low-level file copy operation using read and write system calls.

```
GNU nano 8.7.1 lowlevel_copy.c *
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdlib.h>

int main()
{
    int source, destination;
    char buffer[100];
    int bytesRead;

    source = open("source.txt", O_RDONLY);

    if (source < 0)
    {
        perror("Error opening source file");
        return 1;
    }

    destination = open("copy.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (destination < 0)
    {
        perror("Error creating destination file");
        close(source);
        return 1;
    }

    while ((bytesRead = read(source, buffer, sizeof(buffer))) > 0)
    {
        write(destination, buffer, bytesRead);
    }

    lseek(source, 0, SEEK_SET);

    close(source);
    close(destination);

    printf("File copied successfully using low-level I/O.\n");

    return 0;
}
```

Code

```

ganesh@DESKTOP-79N3772:~/linux_io_practical$ nano lowlevel_copy.c
ganesh@DESKTOP-79N3772:~/linux_io_practical$ echo "This is a Linux low-level file I/O
ganesh@DESKTOP-79N3772:~/linux_io_practical$ cat source.txt
This is a Linux low-level file I/O practical.
ganesh@DESKTOP-79N3772:~/linux_io_practical$

```

Creating a source file

```

ganesh@DESKTOP-79N3772:~/linux_io_practical$ gcc lowlevel_copy.c -o lowlevel_copy
ganesh@DESKTOP-79N3772:~/linux_io_practical$ ls
lowlevel_copy  lowlevel_copy.c  source.txt
ganesh@DESKTOP-79N3772:~/linux_io_practical$

```

Compilation

```

ganesh@DESKTOP-79N3772:~/linux_io_practical$ ./lowlevel_copy
File copied successfully using low-level I/O.
ganesh@DESKTOP-79N3772:~/linux_io_practical$

```

Execute

```

File copied successfully using low-level I/O.
ganesh@DESKTOP-79N3772:~/linux_io_practical$ cat copy.txt
This is a Linux low-level file I/O practical.
ganesh@DESKTOP-79N3772:~/linux_io_practical$ cmp source.txt copy.txt
ganesh@DESKTOP-79N3772:~/linux_io_practical$

```

The files are identical if there is no output

```

ganesh@DESKTOP-79N3772:~/linux_io_practical$ ./lowlevel_copy
File copied successfully using low-level I/O.
ganesh@DESKTOP-79N3772:~/linux_io_practical$ cat copy.txt
This is a Linux low-level file I/O practical.
ganesh@DESKTOP-79N3772:~/linux_io_practical$ cmp source.txt copy.txt
ganesh@DESKTOP-79N3772:~/linux_io_practical$ diff source.txt copy.txt
ganesh@DESKTOP-79N3772:~/linux_io_practical$ ./lowlevel_copy
y.txt
cmp sourceFile copied successfully using low-level I/O.
.txt copy.txtganesh@DESKTOP-79N3772:~/linux_io_practical$ cat copy.txt
This is a Linux low-level file I/O practical.
ganesh@DESKTOP-79N3772:~/linux_io_practical$ cmp source.txt copy.txt
ganesh@DESKTOP-79N3772:~/linux_io_practical$

```

Create program

```
GNU nano 2.7.1 standard_copy.c
#include <stdio.h>
```

```
int main()
{
    FILE *source;
    FILE *destination;
    char buffer[100];
    size_t bytesRead;

    source = fopen("source.txt", "r");

    if (source == NULL)
    {
        perror("Error opening source file");
        return 1;
    }

    destination = fopen("copy_standard.txt", "w");

    if (destination == NULL)
    {
        perror("Error creating destination file");
        fclose(source);
        return 1;
    }
```

```
        perror("Error creating destination file");
        fclose(source);
        return 1;
    }

    while ((bytesRead = fread(buffer, 1, sizeof(buffer), source)) > 0)
    {
        fwrite(buffer, 1, bytesRead, destination);
    }

    fclose(source);
    fclose(destination);

    printf("File copied successfully using standard library I/O.\n");

    return 0;
}
```

```
^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   M-U
```

```

ganesh@DESKTOP-79N3772:~/linux_io_practical$ cmp source.txt copy.txt
ganesh@DESKTOP-79N3772:~/linux_io_practical$ nano standard_copy.c
ganesh@DESKTOP-79N3772:~/linux_io_practical$ gcc standard_copy.c -o standard_copy
ganesh@DESKTOP-79N3772:~/linux_io_practical$ ./standard_copy
File copied successfully using standard library I/O.
ganesh@DESKTOP-79N3772:~/linux_io_practical$ cat copy_standard.txt
This is a Linux low-level file I/O practical.
ganesh@DESKTOP-79N3772:~/linux_io_practical$ time ./lowlevel_copy
File copied successfully using low-level I/O.

real    0m0.004s
user    0m0.001s
sys     0m0.001s
ganesh@DESKTOP-79N3772:~/linux_io_practical$ time ./standard_copy
File copied successfully using standard library I/O.

real    0m0.003s
user    0m0.001s
sys     0m0.003s
ganesh@DESKTOP-79N3772:~/linux io practical$

```

compile and run

Times are different that is computer default. For a very small file, the difference may be extremely small.

Task 2

Develop a program that redirects standard input and output using file descriptors and `dup2()`. Demonstrate how shells implement I/O redirection.

```

#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdlib.h>

int main()
{
    int inputFile;
    int outputFile;

    inputFile = open("input.txt", O_RDONLY);

    if (inputFile < 0)
    {
        perror("Error opening input file");
        return 1;
    }

    outputFile = open("output.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (outputFile < 0)
    {
        perror("Error opening output file");
        close(inputFile);
        return 1;
    }
}

```

```

    perror("Error opening output file");
    close(inputFile);
    return 1;
}

dup2(inputFile, STDIN_FILENO);
dup2(outputFile, STDOUT_FILENO);

close(inputFile);
close(outputFile);

char text[100];

while (fgets(text, sizeof(text), stdin) != NULL)
{
    printf("%s", text);
}

return 0;

```

Create input file

```

ganesh@DESKTOP-79N3772:~/linux_io_practical$ nano redirection.c
ganesh@DESKTOP-79N3772:~/linux_io_practical$ echo "Hello from input file." > input.txt
ganesh@DESKTOP-79N3772:~/linux_io_practical$ echo "This text is redirected using dup2." >> input.txt
ganesh@DESKTOP-79N3772:~/linux_io_practical$ cat input.txt
Hello from input file.
This text is redirected using dup2.
ganesh@DESKTOP-79N3772:~/linux_io_practical$

```

Compile

```

ganesh@DESKTOP-79N3772:~/linux_io_practical$ gcc redirection.c -o redirection
ganesh@DESKTOP-79N3772:~/linux_io_practical$ ls
copy.txt      input.txt    lowlevel_copy.c  redirection.c  standard_copy
copy_standard.txt  lowlevel_copy redirection      source.txt    standard_copy.c
ganesh@DESKTOP-79N3772:~/linux_io_practical$

```

Execute dup()2 program

```

ganesh@DESKTOP-79N3772:~/linux_io_practical$ ./redirection
ganesh@DESKTOP-79N3772:~/linux_io_practical$ cat output.txt
Hello from input file.
This text is redirected using dup2.
ganesh@DESKTOP-79N3772:~/linux_io_practical$

```

Output

Task 1 – Observation

The file was successfully copied using low-level I/O system calls and standard library functions.

Task 2 – Observation

Standard input and output were successfully redirected using `dup2 ()` and file descriptors